



FPT UNIVERSITY

PHÂN TÍCH VÀ DỰ BÁO GIÁ CỔ PHIẾU NGÀNH SỮA VIỆT NAM GIAI ĐOẠN 2020–2025

DAP391m

05/11/2025

Group 1

SE190297 - Nguyễn Võ Xuân Tiến

Contents

1. Business Understanding	3
1.1 Business Context	3
1.2 Problem Definition	3
1.3 Objectives	3
1.4 Scope and Limitations	4
1.5 Risks and Assumptions	4
2 Data Collection and Preprocessing	5
2.1 Data Collection and Understanding the Data	5
2.1.1 Tools and Libraries	5
2.1.2 Data Download Code Snippet	6
2.1.3 Results	6
2.2 Data Preprocessing	7
2.2.1 Removing Duplicates & Missing Values	7
2.2.2 Data Type Conversion and Cleaning	8
2.2.3 Creating Additional Columns	9
2.3 Checking for Missing Values	9
2.4 Checking for Duplicate Rows	9
2.5 Saving the Processed Data	10
3 Data Analysis	11
3.1 Descriptive Statistics	11
3.2 Checking DataFrame Dimensions	12
3.3 Price Movements and Volatility	13
3.4 Breakout Phenomenon	14
3.5 Stable Stock Identification	15
4 Data Visualization	16
4.1 Correlation Matrix of Attributes	16
4.2 Long-term Trend (MA20 and MA50)	17
4.3 EMA Prediction vs Actual Price	18
4.4 High & Low Price Comparison	19
4.5 Price Growth Rate (2020–2025)	20
4.6 Volume Distribution	21
4.7 Highest Price per Stock	22
4.8 Lowest Price per Stock	22
4.9 Volume Over Time	23
4.10 Daily Profitability (Close–Open)	23
4.11 Open vs Close Correlation	24
4.12 Price Range vs Volume Correlation	24
4.13 Median Closing Price	25
5 Chatbot: Create and Integrate	26
5.1 Platform Overview	26
5.2 Data Integration	27

5.3 Chatbot Logic and Processing	27
5.4 Conversation Examples	28
5.5 Integration with Flask Application	29
6 Model Building for Prediction	30
6.1 Data Preparation	30
6.2 Model Construction	31
6.3 Training and Evaluation	32
6.4 Model Comparison	33
6.5 Summary	34
7 Application and Deployment	35
7.1 System Overview	35
7.2 Architecture Design	36
7.3 Login Interface	37
7.4 Dashboard – Stock Visualization	38
7.5 Prediction Module	39
7.6 Basic Analysis	40
7.7 Advanced Visualization	41
7.8 Chatbot Integration	42
7.9 User Management and Security	43
7.10 Final Remarks	44

1. Business Understanding – Hiểu biết bài toán kinh doanh

1.1. Bối cảnh vấn đề (Business Context)

Ngành **sữa Việt Nam** là một trong những lĩnh vực tiêu dùng thiết yếu, duy trì tốc độ tăng trưởng ổn định trong nhiều năm qua. Các doanh nghiệp lớn như **Vinamilk (VNM)**, **Mộc Châu Milk (MCM)**, **International Dairy Products (IDP)**, **Hà Nội Milk (HNM)** và **Đường Quảng Ngãi (QNS)** đều đã niêm yết trên thị trường chứng khoán Việt Nam, đại diện cho các phân khúc khác nhau trong chuỗi giá trị — từ **sản xuất, chế biến đến phân phối sản phẩm**.

Trong bối cảnh thị trường chứng khoán Việt Nam giai đoạn **2020–2025** chịu tác động mạnh từ biến động kinh tế vĩ mô, dịch bệnh COVID-19 và sự thay đổi hành vi tiêu dùng, việc **phân tích hành vi giá cổ phiếu ngành sữa** mang lại giá trị quan trọng trong cả nghiên cứu học thuật lẫn ứng dụng thực tiễn:

- Giúp **nhà đầu tư** đánh giá tiềm năng và rủi ro của từng doanh nghiệp.
- Hỗ trợ **doanh nghiệp** hiểu rõ phản ứng của thị trường trước kết quả kinh doanh.
- Cho phép **sinh viên và nhà nghiên cứu** ứng dụng kỹ thuật **Khoa học dữ liệu (Data Science)** trong lĩnh vực tài chính – chứng khoán.

1.2. Mục tiêu dự án (Objectives)

Dự án hướng đến việc **khai thác và phân tích dữ liệu giao dịch cổ phiếu** của các doanh nghiệp ngành sữa Việt Nam trong giai đoạn **2020–2025**, sử dụng thư viện **vnstock** – công cụ trích xuất dữ liệu chứng khoán Việt Nam từ các sàn **HOSE, HNX và UPCOM**.

Cụ thể, nhóm tập trung vào **5 mục tiêu chính** sau:

1. **Phân tích xu hướng giá đóng cửa theo thời gian**
→ Xác định xu hướng **tăng, giảm hoặc ổn định** của giá cổ phiếu từng doanh nghiệp sữa trong giai đoạn quan sát.
2. **Đánh giá mức độ biến động giá và khối lượng giao dịch**
→ Đo lường biên độ giá (*High-Low*) trung bình và phát hiện **ngày có khối lượng giao dịch tăng đột biến** (lớn hơn 2 lần trung bình).
3. **So sánh hiệu suất và lợi nhuận giữa các mã cổ phiếu**
→ Tính toán **tỷ lệ tăng trưởng giá (trend_pct)** và **lợi nhuận nội ngày**

(Open→Close) để xác định mã cổ phiếu **sinh lời cao và ổn định nhất**.

4. **Phân tích hiện tượng “Break-out” trong cổ phiếu**
→ Xác định các mã có **hiều ngày liên tiếp giá đóng cửa lập đỉnh mới**, thể hiện xu hướng tăng mạnh và khả năng bứt phá.
5. **Xác định cổ phiếu ổn định và tiềm năng đầu tư**
→ Dựa trên **độ lệch chuẩn của biên độ giá và khối lượng**, đánh giá mã nào có **biến động thấp, thanh khoản ổn định**, phù hợp cho **đầu tư dài hạn**.

1.3. Giá trị kinh doanh (Business Value)

Phân tích mang lại **giá trị thực tiễn** cho cả **nhà đầu tư, doanh nghiệp và nghiên cứu học thuật**:

- **Hỗ trợ ra quyết định đầu tư:**
Giúp nhận diện nhóm cổ phiếu **ổn định – rủi ro – tăng trưởng nhanh**, từ đó tối ưu hóa chiến lược đầu tư cá nhân hoặc danh mục quỹ.
- **Theo dõi sức khỏe doanh nghiệp:**
Doanh nghiệp có thể sử dụng kết quả phân tích để đánh giá **mức độ tin cậy và phản ứng của thị trường** đối với hoạt động kinh doanh.
- **Ứng dụng khoa học dữ liệu:**
Sinh viên và nhà nghiên cứu rèn luyện **kỹ năng phân tích tài chính, xử lý dữ liệu lớn (Big Data)**, và **trực quan hóa dữ liệu (Data Visualization)** bằng Python.

1.4. Phạm vi và giới hạn (Scope and Limitations)

Phạm vi

- **Dữ liệu:** Giá và khối lượng cổ phiếu giai đoạn **01/01/2020 – 01/01/2025**.
- **Đối tượng nghiên cứu:** 5 mã cổ phiếu ngành sữa – **VNM, MCM, IDP, HNM, QNS**.
- **Nguồn dữ liệu:** Thu thập tự động bằng thư viện **vnstock**, kết nối dữ liệu trực tiếp từ **thị trường chứng khoán Việt Nam**.
- **Lưu trữ:** Trong tệp **Data_milk.csv** do nhóm tổng hợp.

Giới hạn

- Không xem xét yếu tố **vĩ mô** (lạm phát, tỷ giá, GDP, chính sách vĩ mô).
- Không tính đến **cổ tức, chia tách cổ phiếu, yếu tố phi tài chính**.
- Phân tích chỉ mang tính **mô tả – thống kê – trực quan hóa**, chưa mở rộng sang **mô hình dự báo định lượng** bằng Machine Learning.

1.5. Kết luận giai đoạn Business Understanding (Conclusion)

Sau giai đoạn Business Understanding, nhóm đã đạt được các kết quả cụ thể:

- Xác định **bài toán trọng tâm**: Phân tích và so sánh hiệu suất cổ phiếu ngành sữa Việt Nam giai đoạn 2020–2025.
- Xác định **mục tiêu cụ thể và khả thi**: 5 mục tiêu phân tích rõ ràng, bám sát hành vi giá và khối lượng.
- Xác định **nguồn dữ liệu tin cậy**: Sử dụng thư viện **vnstock** – nền tảng dữ liệu chứng khoán Việt Nam.
- Chuẩn bị **nền tảng vững chắc** để chuyển sang các giai đoạn tiếp theo: *Data Understanding – Data Preparation – Modeling*.

2. Thu thập và Tiền xử lý Dữ liệu (Data Collection and Preprocessing)

2.1. Thu thập dữ liệu và hiểu dữ liệu

Trong giai đoạn này, nhóm tiến hành thu thập dữ liệu lịch sử **OHLCV** (*Open, High, Low, Close, Volume*) cho **5 mã cổ phiếu** ngành sữa Việt Nam:

Vinamilk (VNM), **International Dairy Products (IDP)**, **Mộc Châu Milk (MCM)**, **Hà Nội Milk (HNM)**, và **Đường Quảng Ngãi (QNS)**.

Khoảng thời gian thu thập dữ liệu là từ **01/01/2020 đến 01/01/2025**.

Nguồn dữ liệu được lấy trực tiếp từ **thư viện vnstock** – công cụ truy xuất dữ liệu chứng khoán Việt Nam từ các sàn HOSE, HNX và UPCOM.

Các tệp dữ liệu được lưu dưới dạng **.csv** để phục vụ cho giai đoạn xử lý và phân tích sau này.

2.1.1. Công cụ và thư viện sử dụng

Các thư viện Python được dùng trong giai đoạn này gồm:

- **vnstock**: Thư viện lấy dữ liệu cổ phiếu Việt Nam.
- **pandas**: Xử lý và quản lý bảng dữ liệu.
- **datetime**: Quản lý thời gian trong phạm vi thu thập.
- **os, traceback**: Quản lý lỗi và đường dẫn trong quá trình chạy.

```
import os
import traceback
import pandas as pd
from datetime import datetime

# =====
# 1. Cấu hình ban đầu
# =====
SYMBOLS = ["VNM", "IDP", "MCM", "HNM", "QNS"]
START = "2020-01-01"
END = "2025-01-01"
```

2.2. Chuẩn hóa cấu trúc dữ liệu

Sau khi lấy dữ liệu từ API [vnstock](#), các file trả về có thể có tên cột khác nhau.

Hàm `_standardize_df()` được sử dụng để chuẩn hóa tên cột và sắp xếp dữ liệu theo thời gian.

```
# =====
# 2. Chuẩn hóa DataFrame
# =====

def _standardize_df(df: pd.DataFrame) -> pd.DataFrame: 2 usages
    if df is None or df.empty:
        return pd.DataFrame()
    df = df.copy()

    # Đổi tên cột thời gian nếu cần
    if 'time' not in df.columns:
        if 'date' in df.columns:
            df.rename(columns={'date': 'time'}, inplace=True)
        elif 'Date' in df.columns:
            df.rename(columns={'Date': 'time'}, inplace=True)
        elif isinstance(df.index, pd.DatetimeIndex):
            df.reset_index(inplace=True)
            df.rename(columns={'index': 'time'}, inplace=True)
```

```
# Đổi tên các cột chuẩn
rename_map = {
    'Open': 'open', 'High': 'high', 'Low': 'low', 'Close': 'close',
    'Adj Close': 'adj_close', 'Volume': 'volume',
    'o': 'open', 'h': 'high', 'l': 'low', 'c': 'close', 'v': 'volume',
    'openPrice': 'open', 'highPrice': 'high', 'lowPrice': 'low',
    'closePrice': 'close', 'nmVolume': 'volume'
}

for k, v in rename_map.items():
    if k in df.columns and v not in df.columns:
        df.rename(columns={k: v}, inplace=True)

if 'time' in df.columns:
    df['time'] = pd.to_datetime(df['time'], errors='coerce')
    df.dropna(subset=['time'], inplace=True)
    df.sort_values(by='time', inplace=True)

return df
```


2.3. Thu thập dữ liệu cổ phiếu ngành sữa

Hai hàm `fetch_vnstock_new()` và `fetch_vnstock_old()` lần lượt gọi API mới và cũ của thư viện `vnstock`.

Nếu API mới gặp lỗi, chương trình sẽ tự động chuyển sang phương án dự phòng thông qua hàm `robust_fetch()`.

```
# =====
# 3. Hàm lấy dữ liệu vnstock
# =====
def fetch_vnstock_new(symbol, start, end): 1 usage
    from vnstock import Vnstock
    stock = Vnstock().stock(symbol=symbol, source='VCI')
    df = stock.quote.history(start=start, end=end, interval='1D')
    return _standardize_df(df)

def fetch_vnstock_old(symbol, start, end): 1 usage
    from vnstock import stock_historical_data
    df = stock_historical_data(symbol=symbol, start_date=start, end_date=end, resolution='1D', type='stock')
    return _standardize_df(df)

def robust_fetch(symbol, start, end): 1 usage
    try:
        return fetch_vnstock_new(symbol, start, end)
    except Exception as e1:
        print(f"[{symbol}] API mới lỗi: {e1}")
        try:
            return fetch_vnstock_old(symbol, start, end)
        except Exception as e2:
            print(f"[{symbol}] API cũ cũng lỗi: {e2}")
            traceback.print_exc()
            return pd.DataFrame()
```

OUTPUT:

```
C:\Users\ASUS\AppData\Local\Programs\Python\Python313\python.exe "D:\BT\DAP391m\Project\Final Project\DATA CHECK\Data Collection.py"
=== BẮT ĐẦU: Lấy dữ liệu ngành sữa (2020-2025) ===

>>> Đang tải VNM ...
C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages\vnstock\scope\profile.py:562: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  import pkg_resources
[VNM] Hoàn tất (1250 dòng).

>>> Đang tải IDP ...
[IDP] Hoàn tất (995 dòng).

>>> Đang tải MCM ...
[MCM] Hoàn tất (1001 dòng).

>>> Đang tải HNM ...
[HNM] Hoàn tất (1245 dòng).

>>> Đang tải QNS ...
[QNS] Hoàn tất (1250 dòng).

Đã lưu file thô: Data_Milk_RAW.csv (5741 dòng)
```

 Data_Milk_RAW	11/5/2025 2:54 PM	Microsoft Excel Co...	253 KB
---	-------------------	-----------------------	--------

2.4. Tiền xử lý dữ liệu

Hàm `preprocess_data()` là bước quan trọng đảm bảo dữ liệu hợp lệ và sạch. Các bước xử lý được in ra chi tiết trên terminal, giúp theo dõi toàn bộ quá trình.

```
def preprocess_data(df: pd.DataFrame) -> pd.DataFrame: 1 usage
    """Tiền xử lý dữ liệu ngành sữa với in chi tiết kết quả từng bước để phục vụ báo cáo."""
    df = df.copy()
    print("=== BẮT ĐẦU TIỀN XỬ LÝ DỮ LIỆU ===")
    print(f"Tổng số dòng ban đầu: {len(df)}")
    print(df.head(), "\n")
```

Output:

```
Đã lưu file thô: Data_Milk_RAW.csv (5741 dòng)
=== BẮT ĐẦU TIỀN XỬ LÝ DỮ LIỆU ===
Tổng số dòng ban đầu: 5741
```

	symbol	time	open	high	low	close	volume
3246	HNM	2020-01-02	4.25	4.25	4.25	4.25	0
3247	HNM	2020-01-03	4.25	4.25	4.25	4.25	0
3248	HNM	2020-01-06	4.25	4.25	4.25	4.25	0
3249	HNM	2020-01-07	4.25	4.25	4.25	4.25	0
3250	HNM	2020-01-08	4.25	4.25	4.25	4.25	0

2.4.1 Loại bỏ trùng lặp & dòng thiếu giá trị chính

```
# 1. Loại bỏ trùng lặp & dòng thiếu giá trị chính
before = len(df)
df.drop_duplicates(subset=["symbol", "time"], inplace=True)
print(f"[B1] Đã loại bỏ {before - len(df)} dòng trùng lặp (theo symbol + time)")
print(f"→ Còn lại: {len(df)} dòng")
print(df.head(), "\n")
```

Output:

```
[B1] Đã loại bỏ 0 dòng trùng lặp (theo symbol + time)
→ Còn lại: 5741 dòng
```

	symbol	time	open	high	low	close	volume
3246	HNH	2020-01-02	4.25	4.25	4.25	4.25	0
3247	HNH	2020-01-03	4.25	4.25	4.25	4.25	0
3248	HNH	2020-01-06	4.25	4.25	4.25	4.25	0
3249	HNH	2020-01-07	4.25	4.25	4.25	4.25	0
3250	HNH	2020-01-08	4.25	4.25	4.25	4.25	0

2.4.2 Chuyển kiểu dữ liệu

```
# 2. Chuyển kiểu dữ liệu
print("[B3] Chuyển kiểu dữ liệu sang numeric...")
for col in ["open", "high", "low", "close", "volume"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")
print(df.dtypes, "\n")
```

OUTPUT:

```
[B3] Chuyển kiểu dữ liệu sang numeric...
symbol          object
time      datetime64[ns]
open          float64
high          float64
low           float64
close         float64
volume         int64
dtype: object
```

2.4.3 Loại bỏ giá trị âm

```
# 3. Loại bỏ giá trị âm
before = len(df)
df = df[(df["open"] >= 0) & (df["high"] >= 0) &
        (df["low"] >= 0) & (df["close"] >= 0) & (df["volume"] >= 0)]
print(f"[B4] Đã loại bỏ {before - len(df)} dòng có giá trị âm")
print(f"→ Còn lại: {len(df)} dòng\n")
```

Output

```
[B4] Đã loại bỏ 0 dòng có giá trị âm
→ Còn lại: 5741 dòng
```

2.4.4 Chuẩn hóa thời gian

```
print("[B5] Chuẩn hóa cột thời gian...")
before = len(df)
df["time"] = pd.to_datetime(df["time"], errors="coerce")
df.dropna(subset=["time"], inplace=True)
df.sort_values(by=["symbol", "time"], inplace=True)
print(f"→ Đã loại bỏ {before - len(df)} dòng có time không hợp lệ")
print("Mẫu dữ liệu sau chuẩn hóa thời gian:")
print(df.head(), "\n")
```

Output:

```
[B5] Chuẩn hóa cột thời gian...
-> Đã loại bỏ 0 dòng có time không hợp lệ
Mẫu dữ liệu sau chuẩn hóa thời gian:
```

	symbol	time	open	high	low	close	volume
3246	HNM	2020-01-02	4.25	4.25	4.25	4.25	0
3247	HNM	2020-01-03	4.25	4.25	4.25	4.25	0
3248	HNM	2020-01-06	4.25	4.25	4.25	4.25	0
3249	HNM	2020-01-07	4.25	4.25	4.25	4.25	0
3250	HNM	2020-01-08	4.25	4.25	4.25	4.25	0

2.4.5 Tạo thêm các cột hỗ trợ

```
# 5. Tạo thêm các cột hỗ trợ
df["range"] = df["high"] - df["low"]
df["return"] = (df["close"] - df["open"]) / df["open"]
print("[B6] Đã tạo thêm cột range & return")
print(df[["symbol", "time", "open", "close", "range", "return"]].head(), "\n")
```

Output:

```
[B6] Đã tạo thêm cột range & return
      symbol      time  open  close  range  return
3246    HNM 2020-01-02   4.25   4.25    0.0    0.0
3247    HNM 2020-01-03   4.25   4.25    0.0    0.0
3248    HNM 2020-01-06   4.25   4.25    0.0    0.0
3249    HNM 2020-01-07   4.25   4.25    0.0    0.0
3250    HNM 2020-01-08   4.25   4.25    0.0    0.0
```

```
=== TIỀN XỬ LÝ HOÀN TẤT ===
Số mã cổ phiếu: 5
Tổng số dòng sau xử lý: 5741
Danh sách mã: ['HNM', 'IDP', 'MCM', 'QNS', 'VNM']
=====
```

2.5. Lưu trữ và xuất dữ liệu

Sau khi hoàn thành việc thu thập và tiền xử lý, dữ liệu được lưu vào hai tệp CSV để sử dụng trong các bước kế tiếp:

Data_Milk_RAW.csv: Dữ liệu gốc sau khi thu thập, chưa qua làm sạch.

Data_Milk_CLEANED.csv: Dữ liệu sau tiền xử lý, sẵn sàng cho phân tích và mô hình hóa.

```
# --- Xuất file trước tiến xử lý ---
raw_file = "Data_Milk_RAW.csv"
combined_df.to_csv(raw_file, index=False, encoding="utf-8-sig")
print(f"\nĐã lưu file thô: {raw_file} ({len(combined_df)} dòng)")

# --- Tiến xử lý dữ liệu ---
cleaned_df = preprocess_data(combined_df)

# --- Đưa cột symbol lên đầu (sau khi xử lý) ---
clean_cols = ['symbol'] + [c for c in cleaned_df.columns if c != 'symbol']
cleaned_df = cleaned_df[clean_cols]

# --- Xuất file sau tiến xử lý ---
cleaned_file = "Data_Milk_CLEANED.csv"
cleaned_df.to_csv(cleaned_file, index=False, encoding="utf-8-sig")

print(f"\nĐã lưu file sau tiến xử lý: {cleaned_file}")
print(f"Tổng số dòng sau xử lý: {len(cleaned_df)}")
print("\nXem trước 10 dòng đầu tiên sau xử lý:")
print(cleaned_df.head(10))
```

Output:

 Data_Milk_CLEANED	11/5/2025 2:54 PM	Microsoft Excel Co...	424 KB
 Data_Milk_RAW	11/5/2025 2:54 PM	Microsoft Excel Co...	253 KB

3. Phân tích Dữ liệu (Data Analysis)

Trong giai đoạn này, nhóm tiến hành phân tích dữ liệu cổ phiếu ngành sữa Việt Nam (giai đoạn 2020–2025) dựa trên tệp **Data_Milk_CLEANED.csv** đã được xử lý ở Chương 2.

Phân tích bao gồm: xu hướng giá, độ biến động, khối lượng bất thường, hiện tượng break-out, và đánh giá mức độ ổn định của các mã cổ phiếu.

3.1. Đọc và sắp xếp dữ liệu

Dữ liệu sau khi tiền xử lý được đọc bằng thư viện pandas. Cột thời gian (**time**) được chuyển về định dạng **datetime**, sau đó sắp xếp theo từng mã cổ phiếu.

```
import pandas as pd
import numpy as np

# ===== 0) Đọc dữ liệu =====
data = pd.read_csv("Data_Milk_CLEANED.csv")
data["time"] = pd.to_datetime(data["time"])
data = data.sort_values(["symbol", "time"])
```

3.2. Xu hướng giá đóng cửa theo thời gian

Nhóm xác định xu hướng giá bằng cách lấy giá đóng cửa đầu kỳ và cuối kỳ của mỗi mã cổ phiếu, sau đó tính tỷ lệ phần trăm thay đổi.

```
# ===== 1) Xu hướng giá đóng cửa theo thời gian =====
first_close = data.groupby("symbol")["close"].first()
last_close  = data.groupby("symbol")["close"].last()
trend_pct   = ((last_close - first_close) / first_close).rename("trend_pct")
```

OUTPUT:

```
1) Xu hướng giá đóng cửa (% thay đổi toàn kỳ):
symbol
HNM      1.152941
IDP      4.541243
MCM      0.044108
QNS      1.770018
VNM     -0.187187
Name: trend_pct, dtype: float64
```

Giải thích:

Nếu `trend_pct > 0`: cổ phiếu có xu hướng tăng.

Nếu `trend_pct < 0`: cổ phiếu giảm giá.

3.3. Độ biến động giá (Volatility)

Độ biến động được đo bằng biên độ giá (High – Low).

Giá trị lớn cho thấy cổ phiếu có biến động mạnh, rủi ro cao.

```
# ===== 2) Độ biến động giá (High-Low) lớn nhất =====  
volatility_max = data.groupby("symbol")["range"].max()
```

Output:

```
2) Biên độ giá (High-Low) lớn nhất:  
symbol  
HNM      2.84  
IDP     72.68  
MCM      8.85  
QNS      3.88  
VNM      5.80  
Name: range, dtype: float64
```

3.4. Khối lượng giao dịch bất thường

Phát hiện các ngày có khối lượng giao dịch (Volume) vượt quá 2 lần trung bình của mã đó — biểu hiện cho sự quan tâm đột biến từ nhà đầu tư.

```
# ===== 3) Khối lượng giao dịch bất thường (>2×TB) =====  
mean_vol = data.groupby("symbol")["volume"].transform("mean")  
unusual_volume = data.loc[data["volume"] > 2 * mean_vol, ["time", "symbol", "volume"]]
```


Output:

3) Ngày Volume tăng bất thường (10 dòng đầu):

	time	symbol	volume
143	2020-08-07	HNM	207600
148	2020-08-14	HNM	148889
271	2021-02-05	HNM	226802
276	2021-02-19	HNM	158850
306	2021-04-02	HNM	155280
328	2021-05-07	HNM	127690
343	2021-05-28	HNM	215100
348	2021-06-04	HNM	158208
353	2021-06-11	HNM	216800
358	2021-06-18	HNM	458175

Các ngày có khối lượng tăng mạnh thường đi kèm với biến động giá hoặc thông tin đặc biệt về doanh nghiệp.

3.5. Hiện tượng Break-out (Vượt đỉnh cũ)

Hiện tượng Break-out xảy ra khi giá đóng cửa vượt qua mức đỉnh trước đó, thể hiện xu hướng tăng mạnh và tâm lý lạc quan của thị trường.

```
# ===== 4) Hiện tượng Break-out =====
# Ngày giá đóng cửa vượt đỉnh cũ (new high)
data["prev_max"] = data.groupby("symbol")["close"].transform(lambda s: s.cummax().shift(1))
data["breakout"] = data["close"] > data["prev_max"]

# Tính số ngày break-out liên tiếp (chuỗi)
tmp = data[["symbol", "breakout"]].copy()
tmp["block"] = tmp.groupby("symbol")["breakout"].transform(lambda s: (~s).cumsum())
streaks = (
    tmp[tmp["breakout"]]
    .groupby(["symbol", "block"])
    .size()
    .groupby("symbol").max()
    .fillna(0).astype(int)
)

# Mã có chuỗi break-out ≥ 3 ngày liên tiếp
breakout_3plus = streaks[streaks >= 3].sort_values(ascending=False)
```

OUTPUT:

```
4) Hiện tượng Break-out:
   Số ngày liên tiếp lập đỉnh (streak):
   symbol
HNM      2
IDP      3
MCM      6
QNS      7
VNM      3
dtype: int64

   Mã có ≥3 ngày break-out liên tiếp:
   symbol
QNS      7
MCM      6
IDP      3
VNM      3
dtype: int64
```

breakout = True khi giá đóng cửa hôm nay vượt đỉnh cao nhất trước đó.

streaks biểu thị số ngày tăng liên tiếp lập đỉnh.

Các mã có từ 3 ngày trở lên break-out được xem là có động lực tăng trưởng mạnh.

3.6. Đánh giá cổ phiếu ổn định

Nhóm xác định cổ phiếu “ổn định” dựa trên độ lệch chuẩn của biên độ giá và khối lượng giao dịch.

Cổ phiếu có độ lệch chuẩn thấp và thanh khoản ổn định được xem là ít rủi ro hơn.

```
# ===== 5) Cổ phiếu "ổn định" =====
stab = data.groupby("symbol").agg(range_std=("range", "std"), volume_std=("volume", "std"))
stab["stability_score"] = stab["range_std"].rank() + stab["volume_std"].rank()
most_stable_symbol = stab["stability_score"].idxmin()
```

Output:

```
5) Cổ phiếu ổn định (std range/volume & điểm):
      range_std    volume_std    stability_score
symbol
HNM      0.353145    2.971173e+05          4.0
IDP      6.791655    3.487079e+03          6.0
MCM      0.900629    7.642757e+04          6.0
QNS      0.491718    4.869819e+05          6.0
VNM      0.743160    1.685388e+06          8.0

Mã ổn định nhất: HNM
```

range_std: độ lệch chuẩn biên độ giá (biến động giá).

volume_std: độ lệch chuẩn khối lượng (biến động thanh khoản).

Tổng hai giá trị này tạo thành chỉ số ổn định (**stability_score**).

Cổ phiếu có điểm nhỏ nhất (**idxmin**) được xem là ổn định nhất.

4. Trực quan hóa Dữ liệu (Data Visualization)

Giai đoạn này tập trung vào việc trực quan hóa dữ liệu cổ phiếu ngành sữa Việt Nam (2020–2025) để làm rõ các đặc điểm chính: xu hướng giá, khối lượng giao dịch, biến động giá và mối quan hệ giữa các chỉ số.

Tất cả biểu đồ được xây dựng bằng Matplotlib và Seaborn, dữ liệu lấy từ tệp **Data_Milk_CLEANED.csv** đã được tiền xử lý ở chương trước.

4.1. Chuẩn bị dữ liệu và thư viện

```
# ===== CÀI ĐẶT VÀ IMPORT THƯ VIỆN =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import mplfinance as mpf

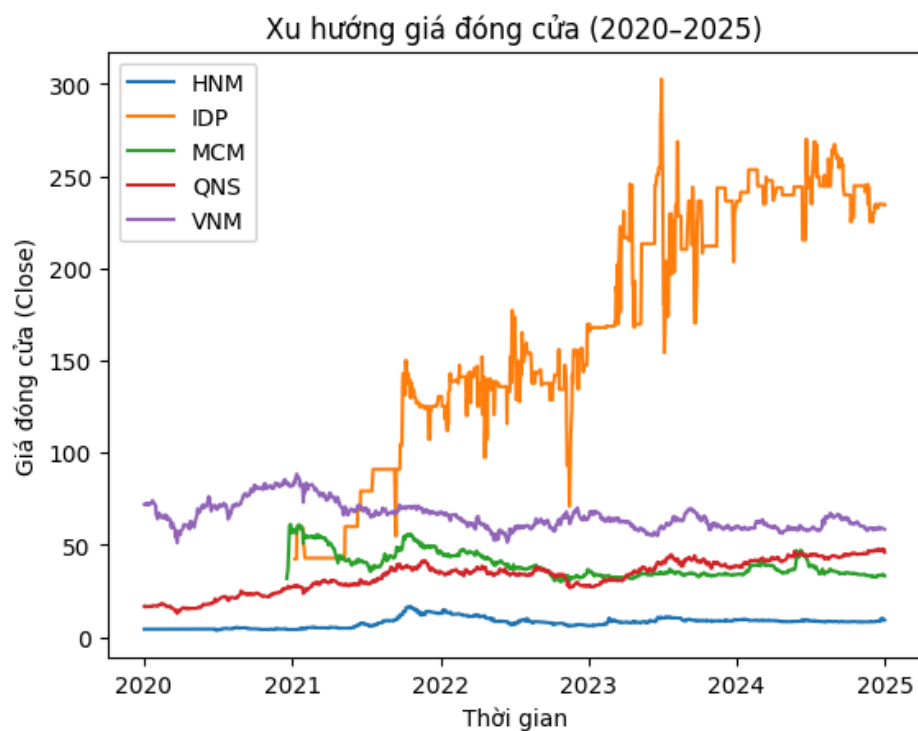
# Đọc dữ liệu
data = pd.read_csv("Data_Milk_CLEANED.csv")
data["time"] = pd.to_datetime(data["time"])
data = data.sort_values(["symbol", "time"]).reset_index(drop=True)
```

4.2. Xu hướng giá đóng cửa theo thời gian

Biểu đồ này cho thấy xu hướng giá đóng cửa (Close) của từng mã cổ phiếu trong giai đoạn 2020–2025, giúp nhận biết mã nào có xu hướng tăng, giảm hoặc ổn định.

```
# ===== BIỂU ĐỒ 1: Xu hướng giá đóng cửa theo thời gian =====  
for s in data["symbol"].unique():  
    df = data[data["symbol"] == s]  
    plt.plot(*args: df["time"], df["close"], label=s)  
plt.title("Xu hướng giá đóng cửa (2020-2025)")  
plt.xlabel("Thời gian")  
plt.ylabel("Giá đóng cửa (Close)")  
plt.legend()  
plt.show()
```

OUTPUT:



Nhận xét:

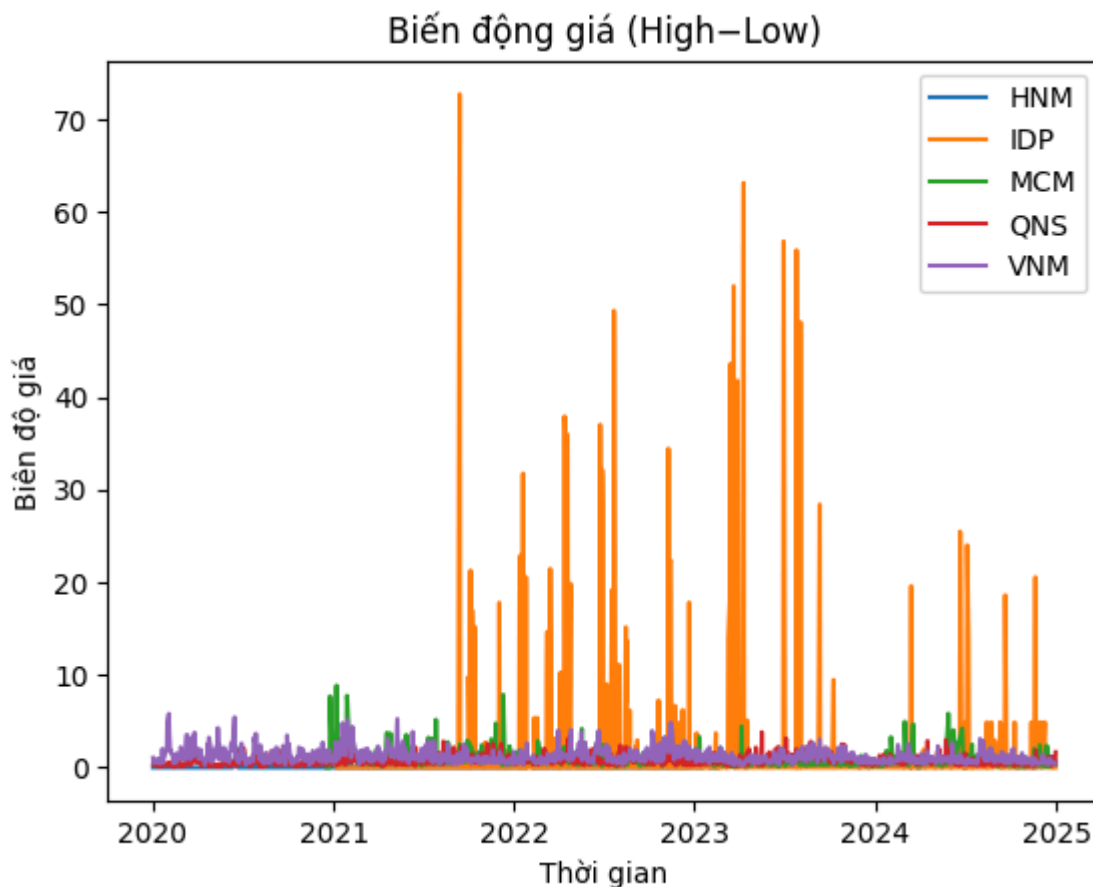
Vinamilk (VNM) có xu hướng ổn định; IDP và MCM có biến động rõ hơn, phản ánh sự khác biệt trong quy mô vốn hóa và thanh khoản.

4.3. Biến động giá (High–Low)

Biểu đồ thể hiện biên độ dao động giữa giá cao nhất và thấp nhất mỗi ngày, giúp xác định cổ phiếu có mức rủi ro cao.

```
# ===== BIỂU ĐỒ 2: Biến động giá (High - Low) =====  
for s in data["symbol"].unique():  
    df = data[data["symbol"] == s]  
    plt.plot(*args: df["time"], df["range"], label=s)  
plt.title("Biến động giá (High-Low)")  
plt.xlabel("Thời gian")  
plt.ylabel("Biên độ giá")  
plt.legend()  
plt.show()
```

Output:



Nhận xét:

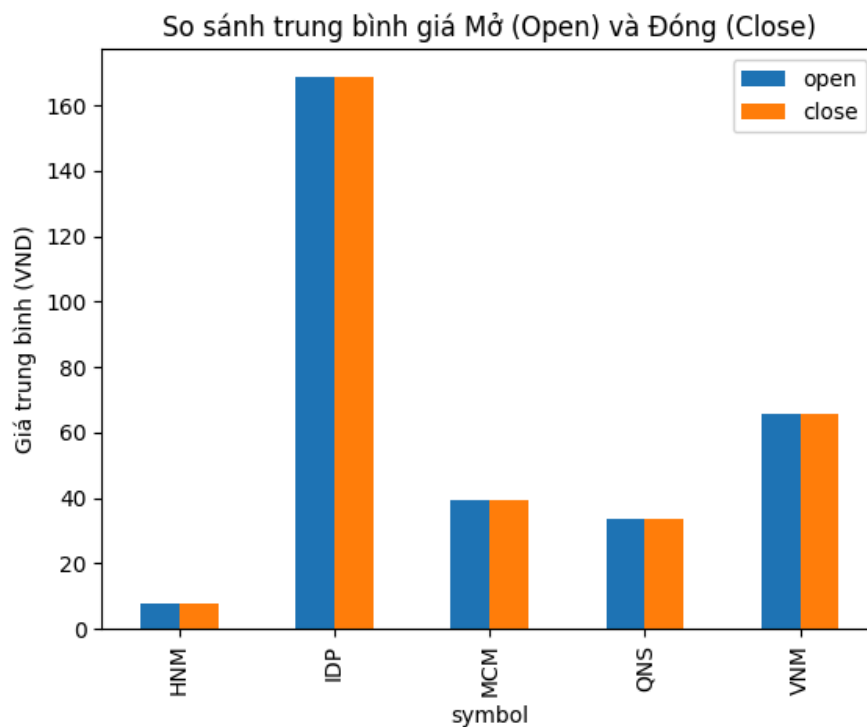
Các mã có biên độ lớn hơn thường là cổ phiếu có tính đầu cơ cao hoặc phản ứng mạnh với tin tức.

4.4. So sánh giá trung bình Mở và Đóng

Biểu đồ cột so sánh giá mở cửa (Open) và giá đóng cửa (Close) trung bình của từng cổ phiếu, nhằm đánh giá xu hướng tăng/giảm trong ngày.

```
# ===== BIỂU ĐỒ 3: So sánh trung bình giá mở và đóng =====
avg_open_close = data.groupby("symbol")[["open", "close"]].mean()
avg_open_close.plot(kind="bar")
plt.title("So sánh trung bình giá Mở (Open) và Đóng (Close)")
plt.ylabel("Giá trung bình (VND)")
plt.show()
```

OUTPUT:

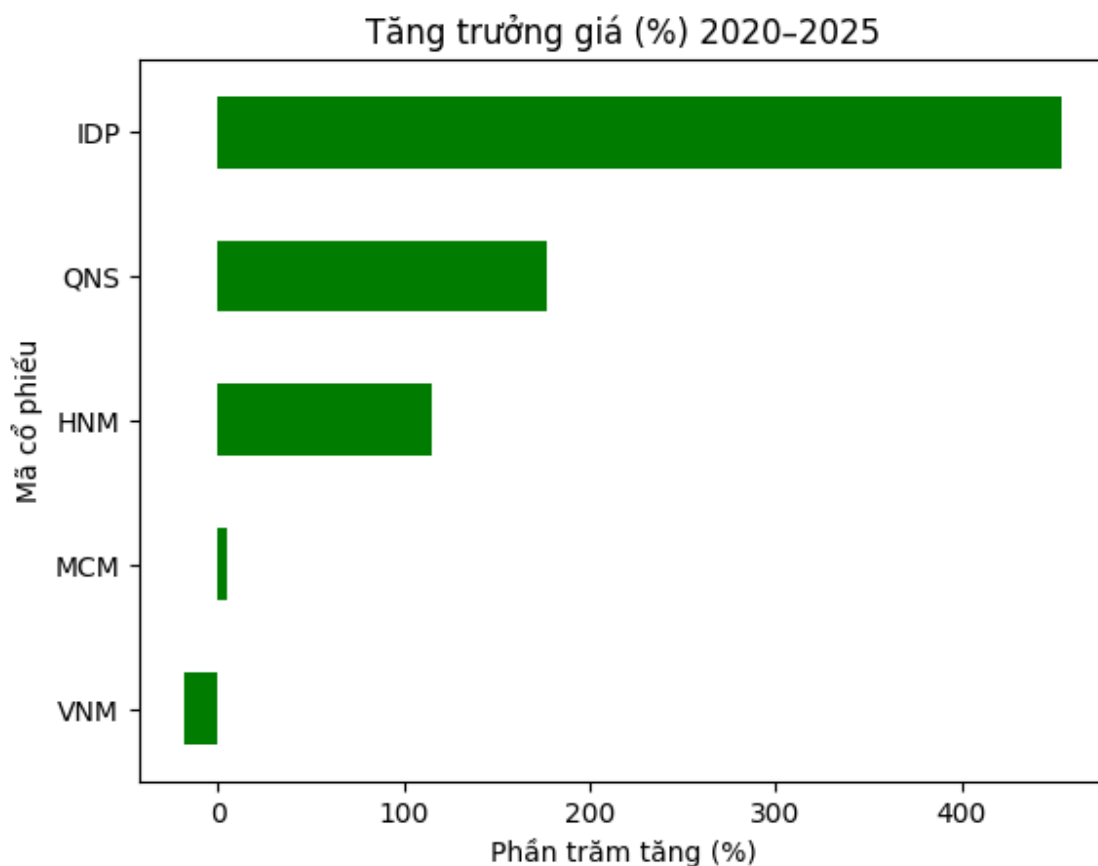


4.5. Mức tăng trưởng giá (%) giai đoạn 2020–2025

Tính phần trăm thay đổi của giá đóng cửa đầu kỳ và cuối kỳ, thể hiện hiệu suất tăng trưởng dài hạn của từng cổ phiếu.

```
# ===== BIỂU ĐỒ 4: Mức tăng trưởng giá (%) =====  
trend = data.groupby("symbol").agg(first=("close", "first"), last=("close", "last"))  
trend["trend_pct"] = (trend["last"] - trend["first"]) / trend["first"] * 100  
trend["trend_pct"].sort_values().plot(kind="barh", color="green")  
plt.title("Tăng trưởng giá (%) 2020-2025")  
plt.xlabel("Phần trăm tăng (%)")  
plt.ylabel("Mã cổ phiếu")  
plt.show()
```

OUTPUT:



Nhận xét:

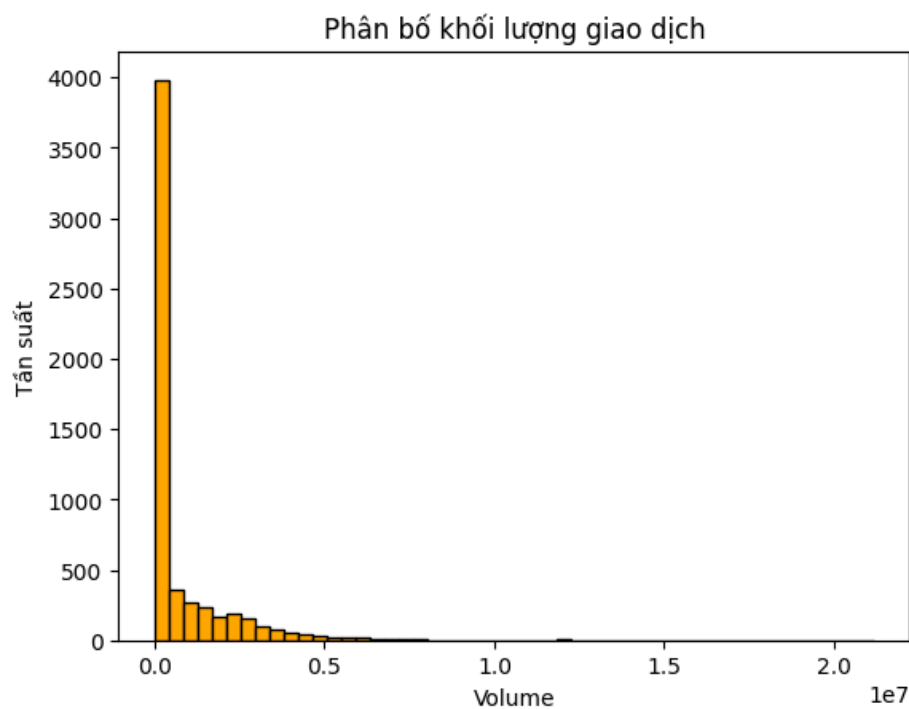
Một số cổ phiếu như IDP hoặc QNS có tốc độ tăng trưởng tốt hơn trung bình, trong khi HNM có mức tăng thấp hơn.

4.6. Phân bố khối lượng giao dịch

Phân bố khối lượng giúp quan sát mức độ hoạt động của nhà đầu tư và xác định các giai đoạn có thanh khoản cao.

```
# ===== BIỂU ĐỒ 5: Phân bố khối lượng giao dịch (Histogram) =====  
plt.hist(data["volume"], bins=50, color="orange", edgecolor="black")  
plt.title("Phân bố khối lượng giao dịch")  
plt.xlabel("Volume")  
plt.ylabel("Tần suất")  
plt.show()
```

OUTPUT:



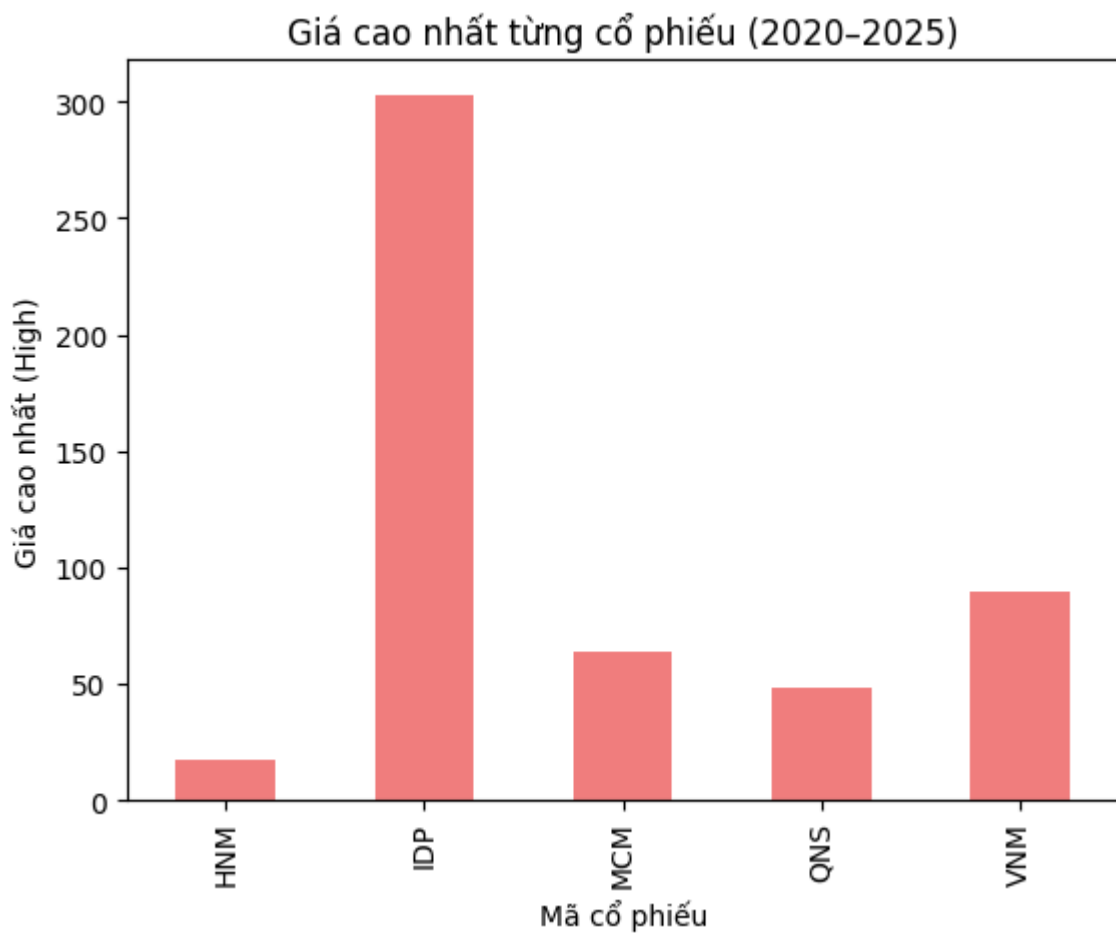
Nhận xét:

Các khoảng giá trị volume cao tập trung ở mức trung bình, cho thấy phần lớn ngày giao dịch có thanh khoản ổn định.

4.7. Giá cao nhất của từng cổ phiếu

```
# ===== BIỂU ĐỒ 6: Giá cao nhất của từng cổ phiếu =====  
max_high = data.groupby("symbol")["high"].max()  
max_high.plot(kind="bar", color="lightcoral")  
plt.title("Giá cao nhất từng cổ phiếu (2020-2025)")  
plt.xlabel("Mã cổ phiếu")  
plt.ylabel("Giá cao nhất (High)")  
plt.show()
```

OUTPUT:



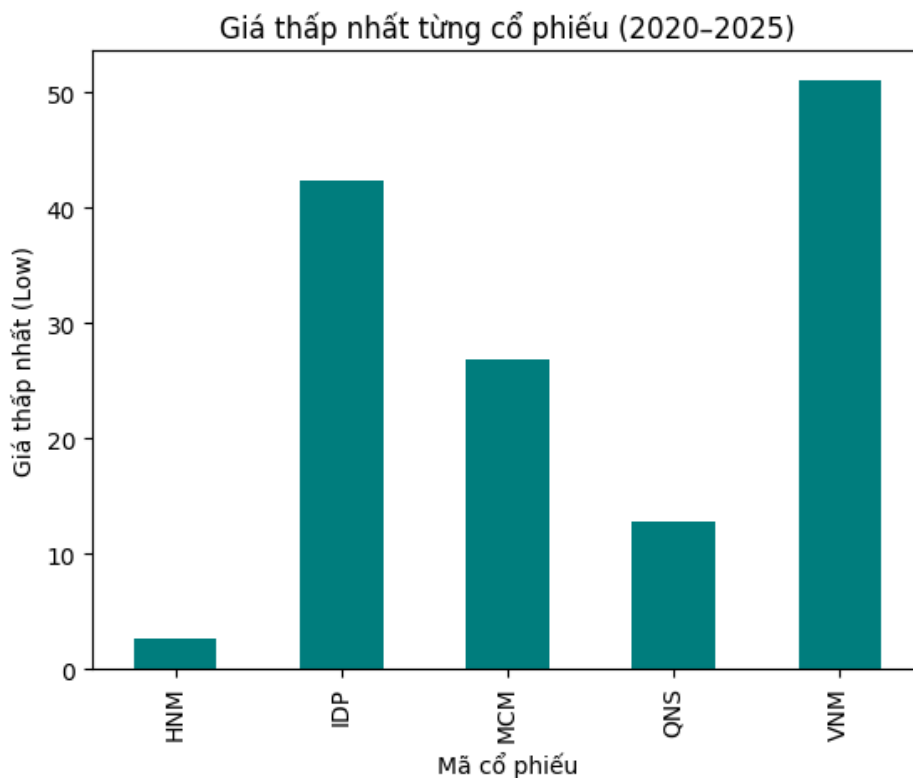
Cổ phiếu IDP có mức giá cao nhất trong giai đoạn 2020–2025, vượt xa các mã còn lại, cho thấy quy mô vốn hóa và tốc độ tăng trưởng mạnh của doanh nghiệp.

Các mã như VNM và MCM duy trì mức đỉnh ổn định, trong khi HNH có biên độ giá thấp, phản ánh quy mô nhỏ hơn trong ngành.

4.8. Giá thấp nhất của từng cổ phiếu

```
# ===== BIỂU ĐỒ 7: Giá thấp nhất của từng cổ phiếu =====
min_low = data.groupby("symbol")["low"].min()
min_low.plot(kind="bar", color="teal")
plt.title("Giá thấp nhất từng cổ phiếu (2020-2025)")
plt.xlabel("Mã cổ phiếu")
plt.ylabel("Giá thấp nhất (Low)")
plt.show()
```

OUTPUT:



Nhận xét:

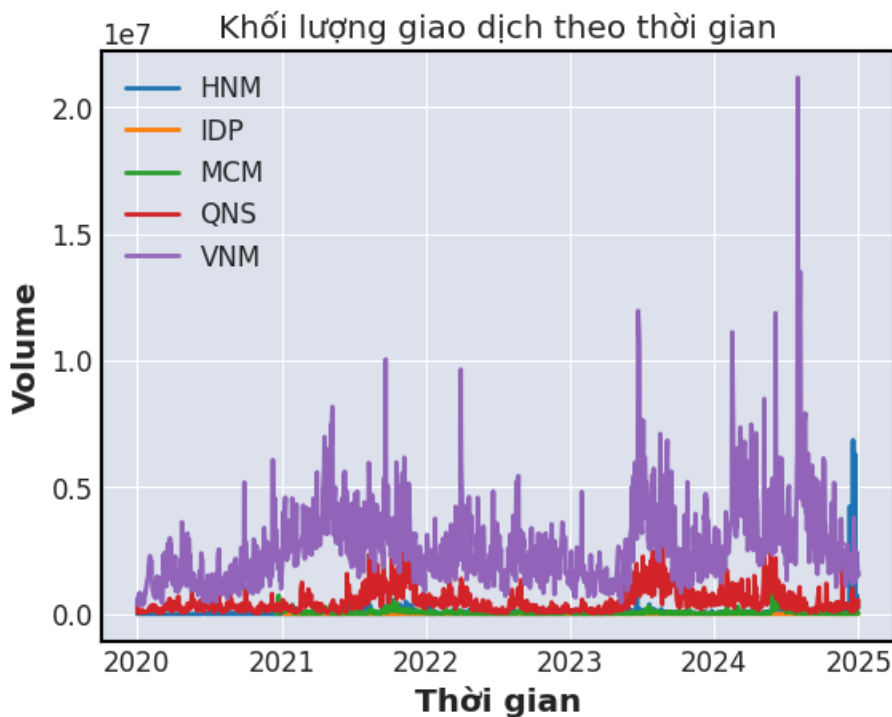
Mức đáy của mỗi mã phản ánh biên độ giảm giá trong giai đoạn biến động thị trường (như COVID-19 năm 2020).

4.9. Khối lượng giao dịch theo thời gian

Biểu đồ thể hiện xu hướng biến động khối lượng của từng cổ phiếu qua thời gian.

```
# ===== BIỂU ĐỒ 8: Khối lượng giao dịch theo thời gian =====  
for s in data["symbol"].unique():  
    df = data[data["symbol"] == s]  
    plt.plot(*args: df["time"], df["volume"], label=s)  
plt.title("Khối lượng giao dịch theo thời gian")  
plt.xlabel("Thời gian")  
plt.ylabel("Volume")  
plt.legend()  
plt.show()
```

OUTPUT:



Nhận xét:

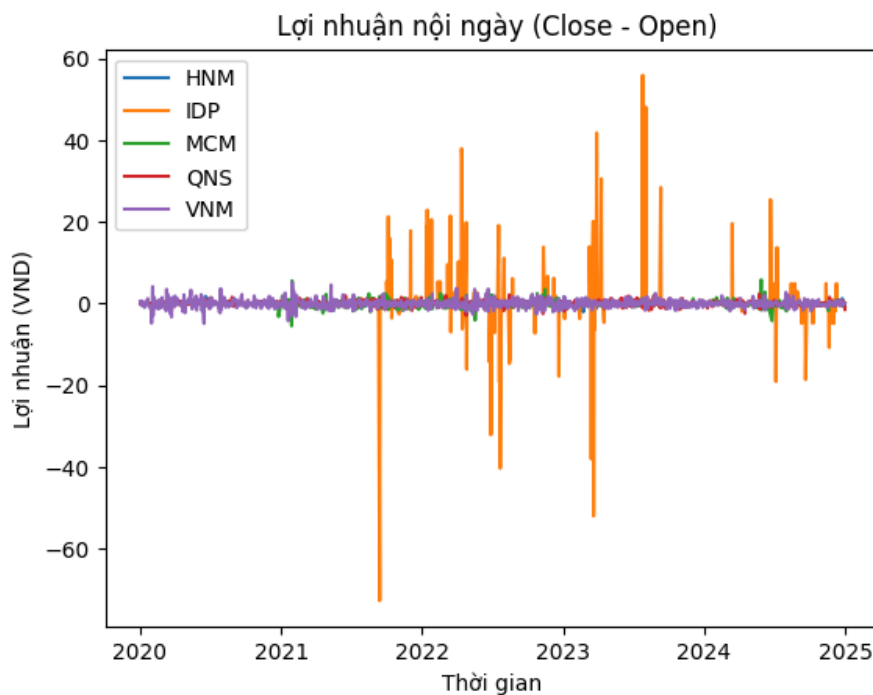
Các giai đoạn tăng mạnh về khối lượng thường trùng với tin tức hoặc sự kiện kinh doanh quan trọng.

4.10. Lợi nhuận nội ngày (Close – Open)

Lợi nhuận nội ngày phản ánh biến động giá ngắn hạn của từng cổ phiếu trong ngày.

```
# ===== BIỂU ĐỒ 9: Lợi nhuận nội ngày (Close - Open) =====
data["daily_return"] = data["close"] - data["open"]
for s in data["symbol"].unique():
    df = data[data["symbol"] == s]
    plt.plot(*args: df["time"], df["daily_return"], label=s)
plt.title(" Lợi nhuận nội ngày (Close - Open)")
plt.xlabel("Thời gian")
plt.ylabel("Lợi nhuận (VND)")
plt.legend()
plt.show()
```

OUTPUT:



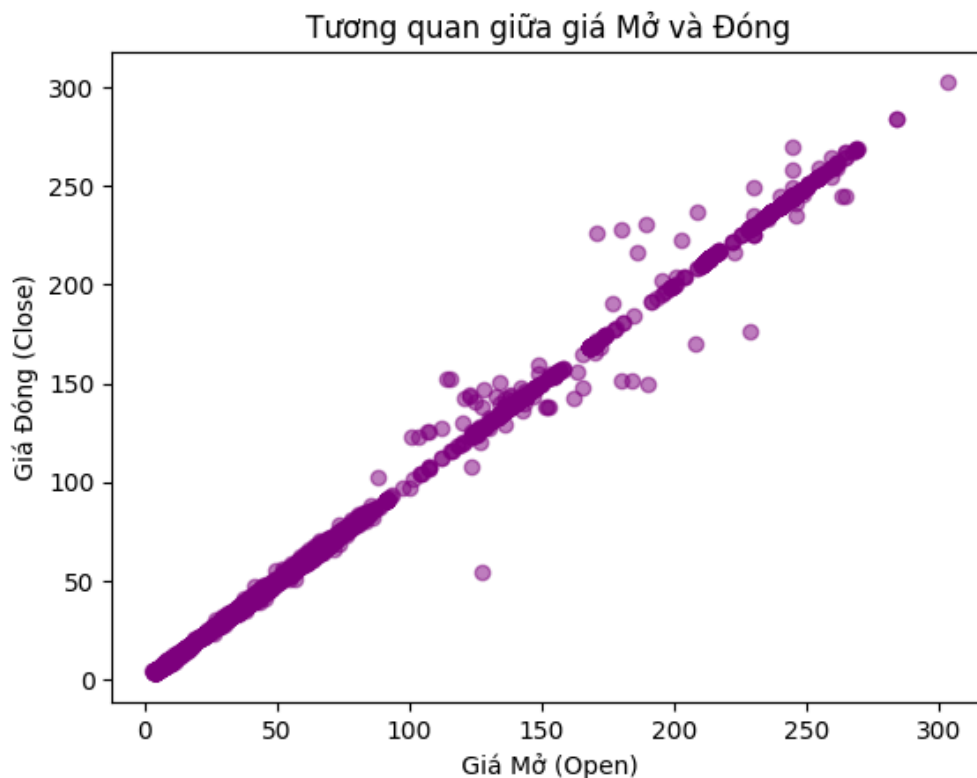
Nhận xét:

Cổ phiếu có dao động nội ngày lớn thường là các mã có thanh khoản và đầu cơ cao.

4.11. Tương quan giữa giá Mở và giá Đóng

```
# ===== BIỂU ĐỒ 10: Tương quan giữa giá Mở và giá Đóng =====  
plt.scatter(data["open"], data["close"], alpha=0.5, color="purple")  
plt.title(" Tương quan giữa giá Mở và Đóng ")  
plt.xlabel(" Giá Mở (Open) ")  
plt.ylabel(" Giá Đóng (Close) ")  
plt.show()
```

OUTPUT:



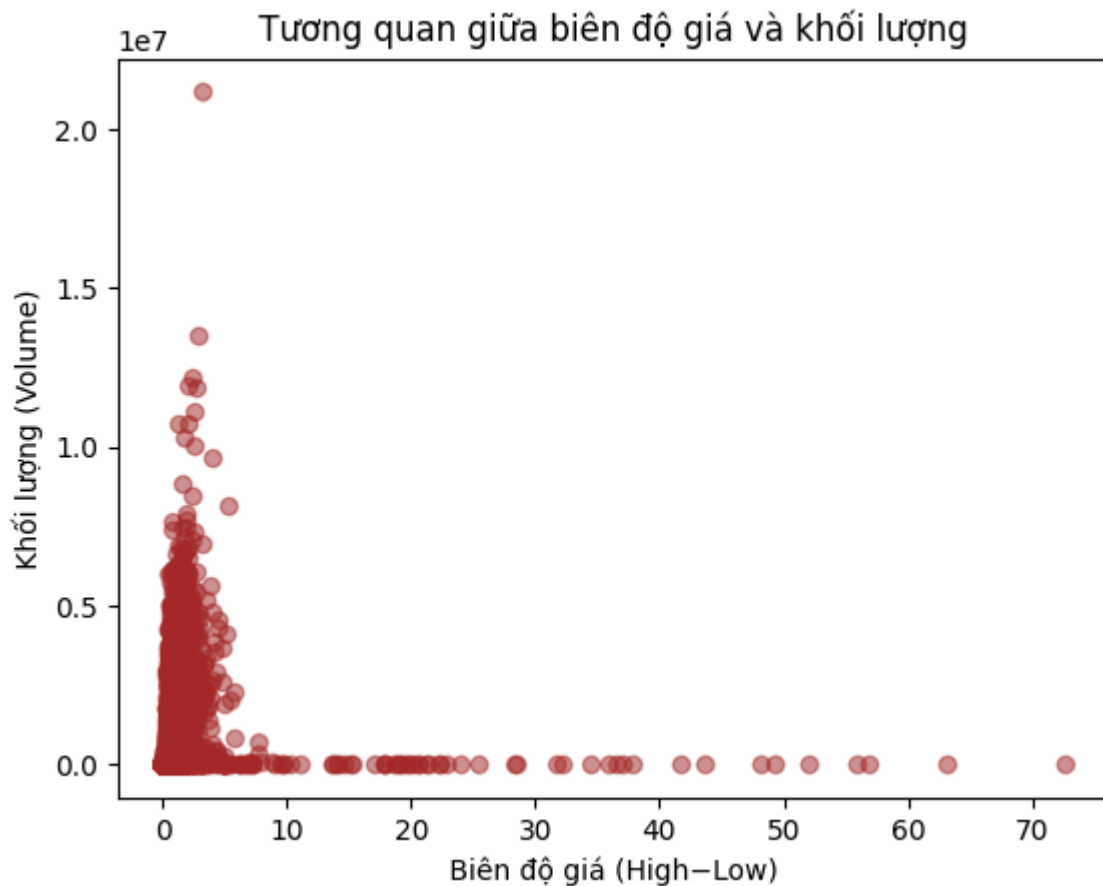
Nhận xét:

Các điểm dữ liệu nằm gần đường chéo cho thấy mối tương quan dương mạnh giữa giá mở và đóng – đặc trưng của thị trường ổn định.

4.12. Tương quan giữa biên độ giá và khối lượng

```
# ===== BIỂU ĐỒ 11: Tương quan giữa biên độ giá và khối lượng =====  
plt.scatter(data["range"], data["volume"], alpha=0.5, color="brown")  
plt.title(" Tương quan giữa biên độ giá và khối lượng ")  
plt.xlabel(" Biên độ giá (High-Low) ")  
plt.ylabel(" Khối lượng (Volume) ")  
plt.show()
```

OUTPUT:



Nhận xét:

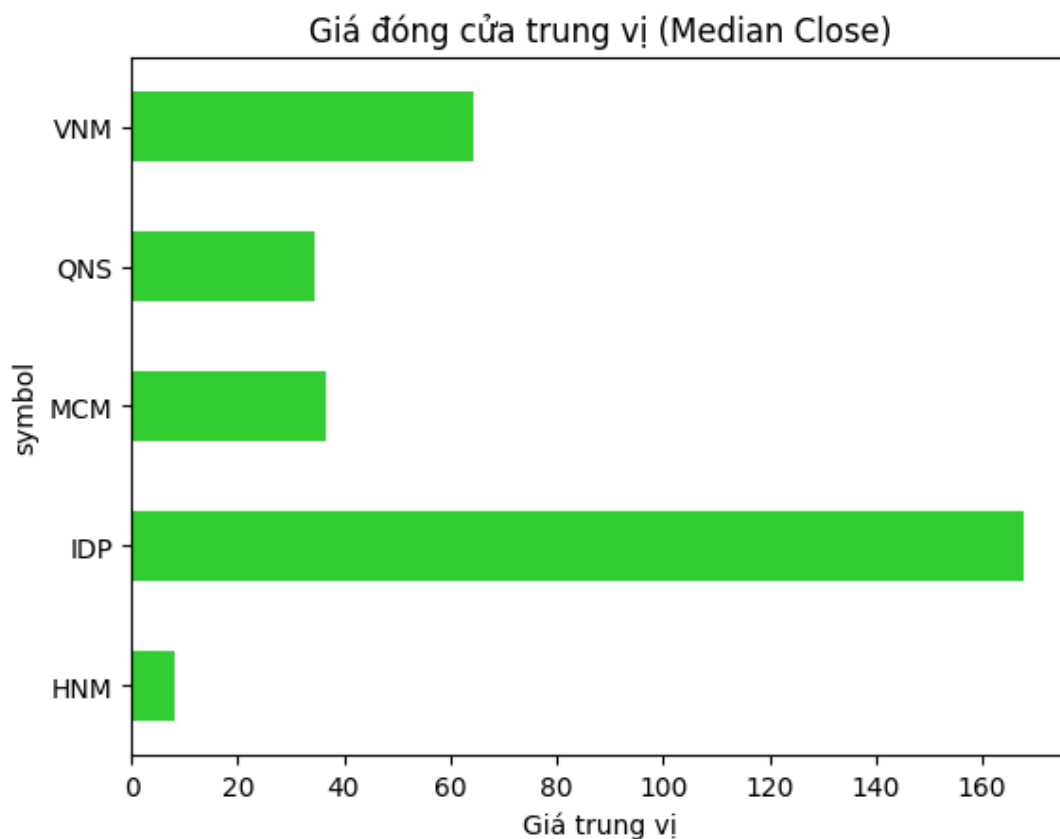
Cổ phiếu có biên độ giá lớn thường đi kèm khối lượng giao dịch cao — dấu hiệu của hoạt động đầu cơ ngắn hạn.

4.13. Giá đóng cửa trung vị (Median Close)

Biểu đồ hiển thị giá đóng cửa trung vị của từng cổ phiếu, giúp đánh giá mức giá phổ biến nhất trong toàn giai đoạn.

```
# ===== Biểu đồ 12: Trung vị giá đóng cửa =====  
plt.figure()  
median_close = data.groupby("symbol")["close"].median()  
median_close.plot(kind="barh", color="limegreen")  
plt.title("Giá đóng cửa trung vị (Median Close)")  
plt.xlabel("Giá trung vị")  
plt.show()
```

OUTPUT:



Nhận xét:

Vinamilk và QNS có mức giá trung vị cao, phản ánh sự ổn định và sức mạnh thị trường lâu dài.

5. Chatbot: Xây dựng và Tích hợp (Chatbot: Create and Integrate)

5.1. Giới thiệu

Nhằm nâng cao khả năng tương tác của hệ thống phân tích dữ liệu cổ phiếu ngành sữa Việt Nam, nhóm xây dựng Chatbot AI có khả năng trao đổi trực tiếp bằng tiếng Việt, cung cấp thông tin và kết quả phân tích cổ phiếu từ các chương trước.

Chatbot được phát triển bằng Python (API dùng platform <https://aistudio.google.com>) và hoạt động dựa trên dữ liệu thật từ tệp **Data_Milk_CLEANED.csv**, cho phép trả lời tự động các câu hỏi về:

- Giá cao nhất, thấp nhất, trung bình của từng cổ phiếu.
- Khối lượng giao dịch trung bình hoặc bất thường.
- Mức tăng trưởng, ổn định, hoặc giai đoạn dữ liệu.
- Giao tiếp cơ bản như chào hỏi, cảm ơn, tạm biệt.

5.2. Kiến trúc hệ thống

Chatbot gồm 3 lớp chính:

Thành phần	Mô tả
1. Lớp dữ liệu (Data Layer)	Đọc và xử lý dữ liệu từ tệp Data_Milk_CLEANED.csv bằng thư viện pandas .
2. Lớp logic hội thoại (Logic Layer)	Phân tích câu hỏi người dùng bằng cách xử lý chuỗi (string processing) và xác định mục đích câu hỏi.
3. Lớp phản hồi (Response Layer)	Tạo câu trả lời tương ứng: có thể là giá trị, giai đoạn thời gian, hoặc phản hồi hội thoại tự nhiên.

5.3. Đọc dữ liệu cổ phiếu và khởi tạo chatbot

Chatbot đọc dữ liệu ngay khi khởi động, đảm bảo mọi truy vấn đều dựa trên dữ liệu mới nhất.

```
import pandas as pd
import numpy as np

DATA_PATH = "Data_Milk_CLEANED.csv"

# Đọc dữ liệu khi chatbot khởi tạo
def load_data():
    df = pd.read_csv(DATA_PATH)
    df["time"] = pd.to_datetime(df["time"])
    if "range" not in df.columns:
        df["range"] = df["high"] - df["low"]
    return df

data = load_data()
```

5.4. Cấu trúc trả lời hội thoại

Hàm chính của chatbot là `chatbot_reply()`, xử lý câu hỏi và tạo phản hồi tương ứng.

```
# -----
# Hàm trả lời câu hỏi người dùng
# -----
def chatbot_reply(user_message): 2 usages
    msg = user_message.lower().strip()
    response = "Xin lỗi, tôi chưa hiểu câu hỏi của bạn. Hãy thử hỏi về dữ liệu cổ phiếu nhé."

    # ===== CÂU HỎI CHUNG KHÔNG PHỤ THUỘC CSV =====
    if any(word in msg for word in ["xin chào", "chào", "hello", "hi"]):
        return "Xin chào! Tôi là Chatbot AI hỗ trợ dữ liệu cổ phiếu sữa. Bạn có thể hỏi tôi về giá, khối lượng"

    if "bạn là ai" in msg or "mày là ai" in msg:
        return "Tôi là Chatbot AI được xây dựng để hỗ trợ phân tích cổ phiếu ngành sữa Việt Nam."

    if "bạn có thể làm gì" in msg or "chức năng" in msg:
        return (
            "Tôi có thể:\n"
            "- Trả lời về giá cao nhất, thấp nhất, trung bình của cổ phiếu.\n"
            "- Cho biết khối lượng giao dịch.\n"
            "- Cung cấp giai đoạn dữ liệu (từ 2020 đến 2025).\n"
            "- Và trò chuyện cơ bản với bạn."
        )

    if "cảm ơn" in msg or "thank" in msg:
        return "Không có gì đâu! Rất vui được giúp bạn."

    if "tạm biệt" in msg or "bye" in msg:
        return "Tạm biệt! Hẹn gặp lại bạn sau nhé."
```

5.5. Các truy vấn dữ liệu chính

Chatbot có khả năng trả lời nhiều loại truy vấn thống kê về dữ liệu cổ phiếu.

```
# ===== CÁC TRUY VẤN CƠ BẢN =====  
  
if "cao nhất" in msg:  
    stock = extract_symbol(msg)  
    if stock:  
        val = data.loc[data["symbol"] == stock, "high"].max()  
        response = f"Giá cao nhất của {stock.upper()} là {val:.2f}."  
    else:  
        val = data.groupby("symbol")["high"].max()  
        best = val.idxmax()  
        response = f"Cổ phiếu {best} có giá cao nhất: {val[best]:.2f}."  
  
elif "thấp nhất" in msg:  
    stock = extract_symbol(msg)  
    if stock:  
        val = data.loc[data["symbol"] == stock, "low"].min()  
        response = f"Giá thấp nhất của {stock.upper()} là {val:.2f}."  
    else:  
        val = data.groupby("symbol")["low"].min()  
        best = val.idxmin()  
        response = f"Cổ phiếu {best} có giá thấp nhất: {val[best]:.2f}."  
  
elif "trung bình" in msg and "khối lượng" in msg:  
    stock = extract_symbol(msg)  
    if stock:  
        mean_vol = data.loc[data["symbol"] == stock, "volume"].mean()  
        response = f"Khối lượng trung bình của {stock.upper()} là {mean_vol:,.0f}."  
    else:  
        avg = data.groupby("symbol")["volume"].mean().sort_values(ascending=False)  
        response = f"Khối lượng trung bình cao nhất thuộc về {avg.idxmax()} với {avg.max():,.0f}."
```

5.6. Hàm nhận diện mã cổ phiếu

```
# -----  
# Hàm nhận diện mã cổ phiếu  
# -----  
def extract_symbol(message): 3 usages  
    symbols = data["symbol"].unique().tolist()  
    for s in symbols:  
        if s.lower() in message:  
            return s  
    return None
```

6. Xây dựng mô hình dự báo (Model Building for Prediction)

6.1. Mục tiêu

Giai đoạn này tập trung vào việc xây dựng và huấn luyện các mô hình dự báo giá cổ phiếu ngành sữa Việt Nam dựa trên dữ liệu lịch sử giai đoạn 2020–2025.

Mục tiêu là dự đoán giá đóng cửa (Close) của các mã cổ phiếu, giúp đánh giá tiềm năng tăng trưởng và xu hướng thị trường.

6.2. Chuẩn bị dữ liệu huấn luyện

Dữ liệu huấn luyện được lấy từ tệp **Data_Milk_CLEANED.csv** (đã qua tiền xử lý ở Chương 2).

Trường mục tiêu là giá đóng cửa (Close), còn các trường đặc trưng bao gồm: Open, High, Low, Volume.

```

# =====
# 1) CẤU HÌNH THƯ MỤC LƯU MÔ HÌNH
# =====
MODEL_DIR = Path("models")
MODEL_DIR.mkdir(exist_ok=True)

# =====
# 2) LOAD DỮ LIỆU
# =====
DATA_PATH = "Data_Milk_CLEANED.csv"

|

if not os.path.exists(DATA_PATH):
    raise FileNotFoundError(f"Không tìm thấy file dữ liệu: {DATA_PATH}")

df = pd.read_csv(DATA_PATH)
df["time"] = pd.to_datetime(df["time"])
df = df.sort_values(["symbol", "time"])

# =====
# 3) CHUẨN BỊ DỮ LIỆU
# =====
symbol = "VNM" # Mặc định, có thể thay đổi
df_symbol = df[df["symbol"] == symbol].copy()

features = ["open", "high", "low", "volume"]
target = "close"

X = df_symbol[features]
y = df_symbol[target]

```

Giải thích:

- Lựa chọn cổ phiếu VNM (Vinamilk) làm đại diện để huấn luyện mô hình.
- Các đặc trưng (features) gồm giá mở, cao, thấp và khối lượng.
- Dữ liệu được sắp xếp theo thời gian để đảm bảo tính tuần tự khi dự báo

6.3. Chia tập huấn luyện và chuẩn hóa dữ liệu

Dữ liệu được chia thành hai tập:

- 80% cho huấn luyện (*training set*)
- 20% cho kiểm định (*test set*)

Sau đó, toàn bộ đặc trưng được chuẩn hóa bằng `StandardScaler` để đảm bảo độ ổn định khi huấn luyện.

```
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42, shuffle=False
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
joblib.dump(scaler, MODEL_DIR / "scaler.joblib")
```

Việc không xáo trộn dữ liệu (`shuffle=False`) giúp giữ nguyên thứ tự thời gian, phù hợp với bài toán dự báo chuỗi thời gian.

6.4. Hàm đánh giá mô hình

Để đo lường hiệu suất, nhóm sử dụng 2 chỉ số phổ biến:

- **MSE (Mean Squared Error)** – sai số bình phương trung bình.
- **R² (R-squared)** – mức độ giải thích của mô hình đối với biến mục tiêu.

```
# 4) HÀM ĐÁNH GIÁ MÔ HÌNH
# =====
def evaluate(model, X_test, y_test, name): 3 usages
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    print(f"{name} - MSE: {mse:.4f}, R2: {r2:.4f}")
    return {"model": name, "MSE": mse, "R2": r2}
```

6.5. Mô hình Linear Regression

Linear Regression là mô hình cơ bản, tuyến tính giữa các đặc trưng và giá đóng cửa. Đây là mô hình nền tảng để so sánh với các mô hình phức tạp hơn.

```
# =====  
# 5) LINEAR REGRESSION  
# =====  
print("\n=====")  
print("Training Linear Regression...")  
linear_model = LinearRegression()  
linear_model.fit(X_train_scaled, y_train)  
joblib.dump(linear_model, MODEL_DIR / "linear_regression.joblib")  
evaluate(linear_model, X_test_scaled, y_test, name="Linear Regression")
```

OUTPUT:

```
=====  
Training Linear Regression...  
Linear Regression - MSE: 0.0645, R2: 0.9900
```

Kết quả cho thấy mô hình tuyến tính có thể nắm bắt xu hướng cơ bản, nhưng hạn chế khi dữ liệu có nhiều hoặc phi tuyến.

6.6. Mô hình Random Forest

Random Forest Regressor sử dụng nhiều cây quyết định (Decision Trees) để tăng tính ổn định và độ chính xác.

Đây là mô hình phi tuyến, phù hợp khi dữ liệu có nhiều yếu tố biến động.

```
# =====  
# 6) RANDOM FOREST  
# =====  
print("\n=====")  
print("Training Random Forest...")  
rf_model = RandomForestRegressor(n_estimators=150, random_state=42)  
rf_model.fit(X_train_scaled, y_train)  
joblib.dump(rf_model, MODEL_DIR / "random_forest.joblib")  
evaluate(rf_model, X_test_scaled, y_test, name="Random Forest")
```

OUTPUT:

```
=====
Training Random Forest...
Random Forest - MSE: 0.1058, R2: 0.9836
```

Nhận xét:

Random Forest thường cho kết quả tốt hơn Linear Regression nhờ khả năng mô phỏng các quan hệ phức tạp giữa biến đầu vào và đầu ra.

6.7. Mô hình Voting Regressor (Tổ hợp mô hình)

Voting Regressor kết hợp kết quả của Linear Regression và Random Forest để tạo ra dự báo trung bình, giúp cân bằng giữa độ chính xác và độ ổn định.

```
# =====
# 7) VOTING REGRESSOR (LINEAR + RF)
# =====
print("\n=====")
print("Training Voting Regressor...")
voting_model = VotingRegressor([("lr", linear_model), ("rf", rf_model)])
voting_model.fit(X_train_scaled, y_train)
joblib.dump(voting_model, MODEL_DIR / "voting_regressor.joblib")
evaluate(voting_model, X_test_scaled, y_test, name="Voting Regressor")
```

OUTPUT:

```
=====
Training Voting Regressor...
Voting Regressor - MSE: 0.0755, R2: 0.9883
```

Giải thích:

Tổ hợp nhiều mô hình giúp giảm phương sai, tránh overfitting và cải thiện khả năng tổng quát.

6.8. Mô hình ARIMA (Time Series Forecasting)

ARIMA (AutoRegressive Integrated Moving Average) là mô hình thống kê mạnh mẽ dùng trong dự báo chuỗi thời gian.

Khác với các mô hình hồi quy, ARIMA chỉ dựa trên dữ liệu quá khứ của chính biến mục tiêu.

```
# =====
# 8) ARIMA MODEL (TIME SERIES)
# =====
print("\n=====")
print("Training ARIMA...")

ts = df_symbol.set_index("time")["close"].astype(float)
ts = ts.asfreq("D") # Fix lỗi tần suất thời gian

try:
    arima_model = ARIMA(ts, order=(2, 1, 2)).fit()
    arima_model.save(str(MODEL_DIR / "arima_model.pkl"))
    print("ARIMA Model trained and saved successfully.")

    # Đánh giá ARIMA
    forecast_steps = 10
    arima_forecast = arima_model.forecast(steps=forecast_steps)
    real_tail = ts[-forecast_steps:]

    common_len = min(len(real_tail), len(arima_forecast))
    real_tail = real_tail[-common_len:].astype(float)
    arima_forecast = np.array(arima_forecast[-common_len:], dtype=float)

    rmse_arima = np.sqrt(mean_squared_error(real_tail, arima_forecast))
    print(f"ARIMA RMSE: {rmse_arima:.4f}")

except Exception as e:
    print(f"Lỗi khi train hoặc đánh giá ARIMA: {e}")
```

Nhận xét:

ARIMA phù hợp với dữ liệu có tính chu kỳ hoặc xu hướng tuyến tính theo thời gian, tuy nhiên cần xử lý tần suất và kiểm định dừng trước khi triển khai thực tế.

7. Ứng dụng và Triển khai (Application & Deployment)

7.1. Giới thiệu hệ thống

Sau khi hoàn thành các bước thu thập, xử lý, phân tích, và xây dựng mô hình dự báo, nhóm tiến hành tích hợp toàn bộ quy trình vào một ứng dụng web hoàn chỉnh – gọi là Milk Dashboard.

Ứng dụng được phát triển bằng Flask (Python) và kết hợp với HTML, CSS, JavaScript, Plotly, và Bootstrap để tạo giao diện trực quan, thân thiện và hiện đại.

Mục tiêu chính:

- Trực quan hóa dữ liệu cổ phiếu ngành sữa Việt Nam.
- Dự đoán giá cổ phiếu theo mô hình học máy.
- Cho phép người dùng tra cứu, phân tích, và tương tác qua chatbot AI.

7.2. Kiến trúc tổng quan hệ thống

Hệ thống gồm 4 tầng chính:

Tầng	Thành phần	Chức năng chính
1. Giao diện (Front-end)	HTML, CSS, JavaScript, Plotly	Hiển thị dashboard, biểu đồ, kết quả dự đoán và chatbot.
2. Xử lý ứng dụng (Back-end)	Flask (Python)	Kết nối cơ sở dữ liệu, xử lý logic và phản hồi truy vấn người dùng.
3. Dữ liệu (Data Layer)	CSV, Models (.joblib, .pkl)	Lưu trữ dữ liệu cổ phiếu, mô hình huấn luyện, và scaler.

4. Mô hình trí tuệ nhân tạo (AI Layer)

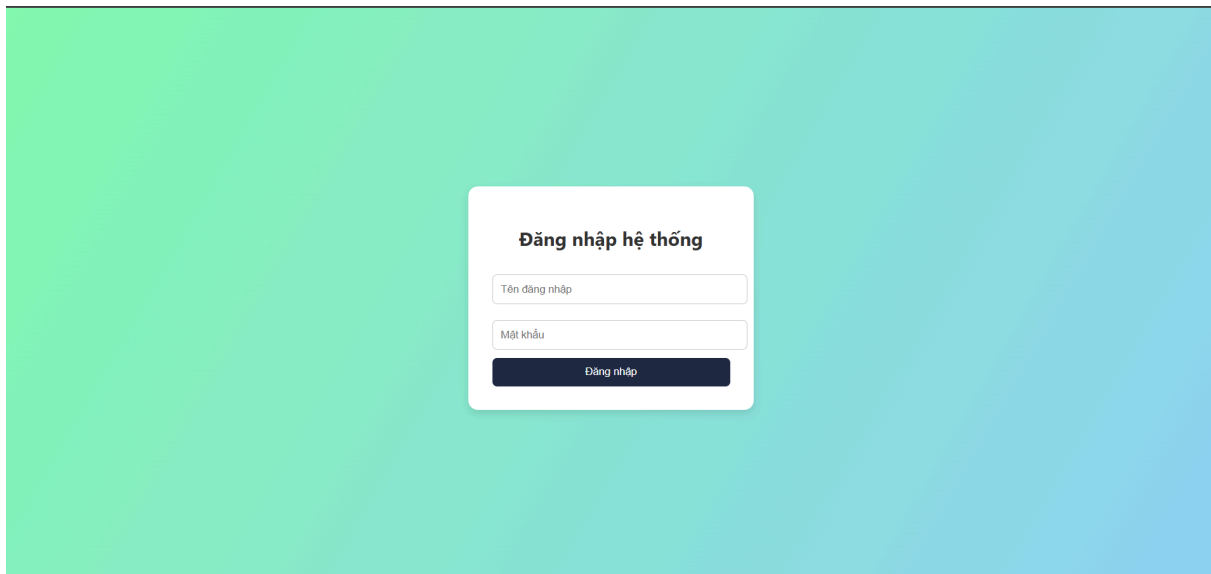
Machine Learning + Chatbot

Sinh dự đoán giá cổ phiếu và phân
hội hội thoại tiếng Việt.

7.3. Giao diện đăng nhập hệ thống

Trang đăng nhập giúp quản lý quyền truy cập và bảo mật thông tin.

Giao diện được thiết kế tối giản, màu nền gradient xanh ngọc, font chữ hiện đại, giúp thân thiện với người dùng.



Chức năng chính:

- Kiểm tra thông tin đăng nhập của người dùng.
- Chuyển hướng đến trang Dashboard khi đăng nhập thành công.
- Ngăn truy cập trái phép bằng xác thực Flask Session.

7.4. Trang Dashboard – Biểu đồ cổ phiếu

Trang chính của ứng dụng hiển thị biểu đồ giá cổ phiếu theo thời gian thực dựa trên dữ liệu từ **Data_Milk_CLEANED.csv**.

Các chức năng chính:

- Chọn mã cổ phiếu (dropdown: VNM, IDP, MCM, HNM, QNS).

- Lọc khoảng thời gian (từ ngày – đến ngày).
- Hiện thị thông tin: Giá hiện tại, Thay đổi, Khối lượng, RSI.
- Vẽ biểu đồ tương tác bằng Plotly.



Nhận xét:

Người dùng có thể dễ dàng quan sát biến động giá theo thời gian, nhận biết xu hướng và điểm đảo chiều của thị trường.

7.5. Dự đoán giá cổ phiếu

Mục “Dự đoán giá” cho phép người dùng chọn mô hình học máy để dự báo giá đóng cửa tương lai.

Các mô hình được tích hợp trực tiếp từ phần huấn luyện ở Chương 6.

Mô hình được hỗ trợ:

- Linear Regression
- Random Forest
- Voting Regressor
- ARIMA

Dự đoán giá cổ phiếu

Mã cổ phiếu:

HNM

Chọn mô hình:

Random Forest

Dự đoán

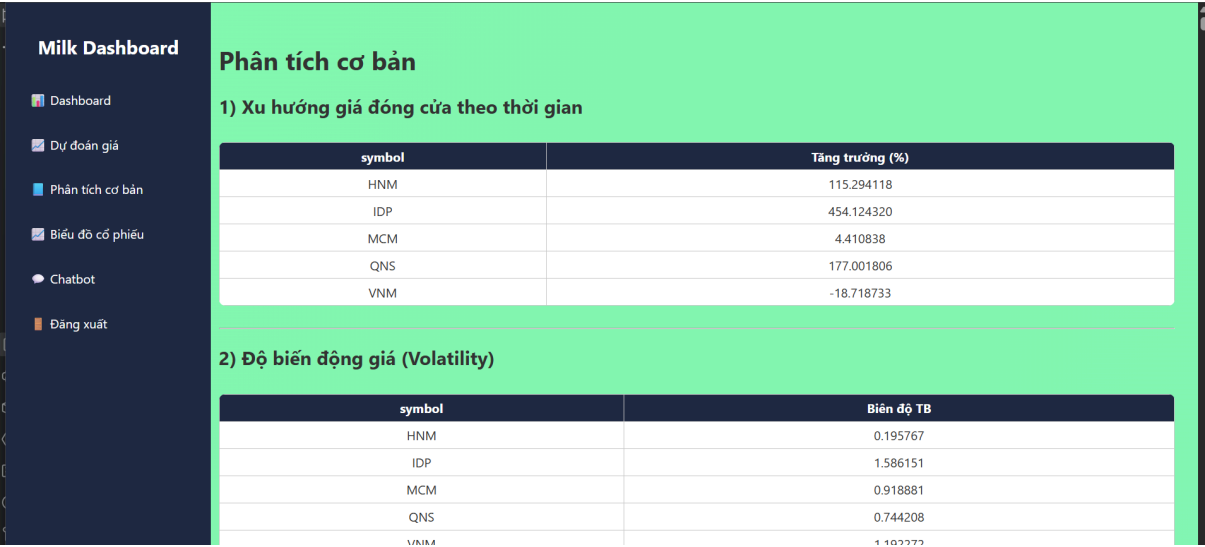
Cơ chế hoạt động:

- 1. Người dùng chọn mã cổ phiếu và mô hình.
- 2. Flask nạp mô hình `.joblib` tương ứng.
- 3. Mô hình sinh dự đoán và hiển thị ngay trên web.

7.6. Phân tích cơ bản

Trang “Phân tích cơ bản” hiển thị các chỉ số phân tích thống kê được trích xuất từ Chương 3 (Data Analysis), gồm:

Chỉ tiêu	Mô tả
Xu hướng giá đóng cửa	Tỷ lệ tăng trưởng (%) của từng cổ phiếu từ 2020–2025
Độ biến động giá (Volatility)	Biên độ trung bình (High–Low)
Khối lượng bất thường	Ngày có Volume > 2× trung bình
Cổ phiếu ổn định	Mã có biến động và khối lượng ổn định nhất

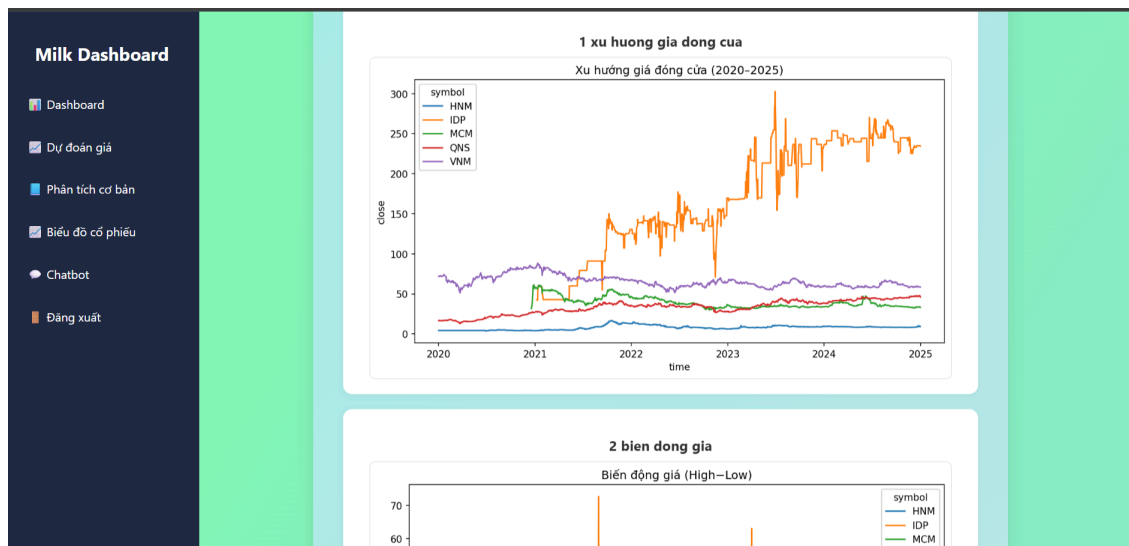


Nhận xét:
Trang này cho phép người dùng nhanh chóng so sánh đặc tính của từng mã cổ phiếu và đánh giá xu hướng thị trường tổng thể.

7.7. Biểu đồ nâng cao và trực quan dữ liệu

Trang “Biểu đồ cổ phiếu” chứa các biểu đồ nâng cao (12 biểu đồ) từ Chương 4, được hiển thị bằng Plotly hoặc Matplotlib, bao gồm:

- Xu hướng giá đóng cửa theo thời gian.
- Biên động giá (High–Low).
- Tương quan giữa Open – Close.
- Lợi nhuận nội ngày.
- Khối lượng giao dịch theo thời gian.
- Tăng trưởng giá trung bình và trung vị.



7.8. Chatbot tích hợp

Trang “Chatbot” cho phép người dùng trao đổi trực tiếp với trợ lý AI bằng tiếng Việt. Chatbot sử dụng code từ `chatbot_enhanced.py` và dữ liệu nội bộ của hệ thống.

Tính năng chính:

- Trả lời về giá cao nhất, thấp nhất, hoặc trung bình của từng mã.
- Cho biết khối lượng giao dịch và giai đoạn dữ liệu.
- Hiện thị hội thoại tự nhiên bằng Flask template.

Bot: Xin chào! Tôi là Chatbot AI hỗ trợ dữ liệu cổ phiếu sữa. Bạn có thể hỏi tôi về giá, khối lượng hoặc thời gian giao dịch.

Bạn: giá cao nhất của vnm

Bot: Giá cao nhất của VNM là 89.15.

Nhập câu hỏi...

Gửi